



Bharat Ratna Indira Gandhi College of Engineering, Solapur

Master Of Computer Application

**Subject: Data Structure and Algorithms
(MCAC102)**

Semester-I

Unit 4: Trees and Algorithms

1. Basic Terminology of Trees

A. Introduction to Trees

1. Definition and Structure of a Tree

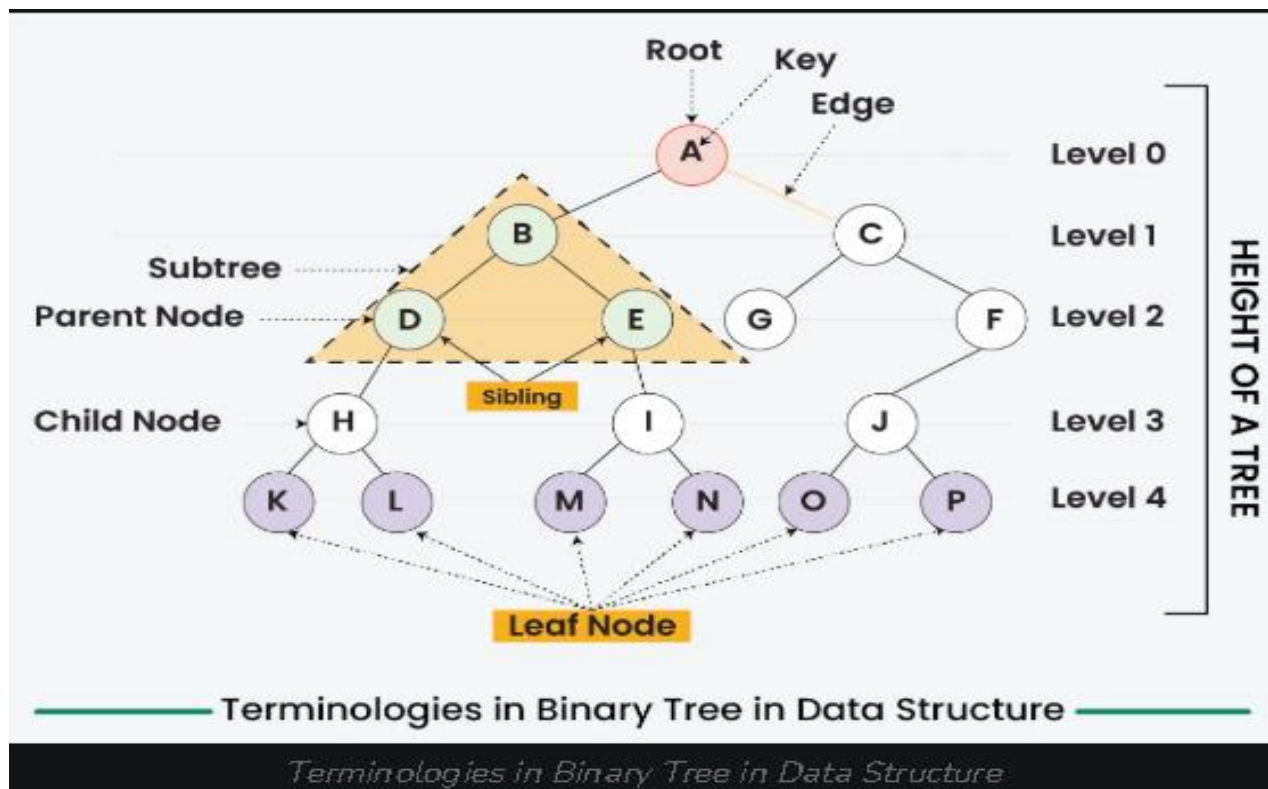
A **Tree** is a connected, acyclic graph with a unique path between any two nodes. In more specific terms:

Sr.NO	Feature	Trees	Linear Data Structures (Arrays, Linked Lists)
1	Structure Type	Hierarchical	Linear
2	Data Organization	Non-linear, branching	Linear, sequential
3	Access Method	Traversal methods (Preorder, In order, Post order, Level-order)	Sequential access (Array index, Linked List nodes)
4	Storage	Parent-child relationships between nodes	Continuous memory (Array) or pointer-based links (Linked List)
5	Insertion/Deletion	Depends on tree type; generally, $O(\log n)$ for balanced trees	Easy in Linked Lists ($O(1)$ for head/tail), but costly in Arrays ($O(n)$)
6	Access Time	Typically, $O(\log n)$ in balanced trees	$O(1)$ for Arrays; $O(n)$ for Linked Lists
7	Dynamic Size	Can dynamically grow and shrink	Arrays are fixed-size; Linked Lists can grow dynamically
8	Path Existence	Paths exist between nodes in levels	No paths; each element has a single predecessor and successor
9	Traversal Complexity	Varies (usually $O(n)$ for full traversal)	$O(n)$ for Linked Lists; $O(1)$ for random access Arrays
10	Types	Binary Tree, AVL Tree, B-Tree, etc.	Simple types like Array and Linked List

11	Searching	Fast in balanced trees ($O(\log n)$)	$O(n)$ in Linked Lists; $O(\log n)$ in sorted Arrays
12	Hierarchy Representation	Naturally represents parent-child relationships	Cannot represent hierarchical relationships directly
13	Applications	Used in databases, compilers, AI (decision trees), file systems	Used in simpler data structures, memory management, and indexing

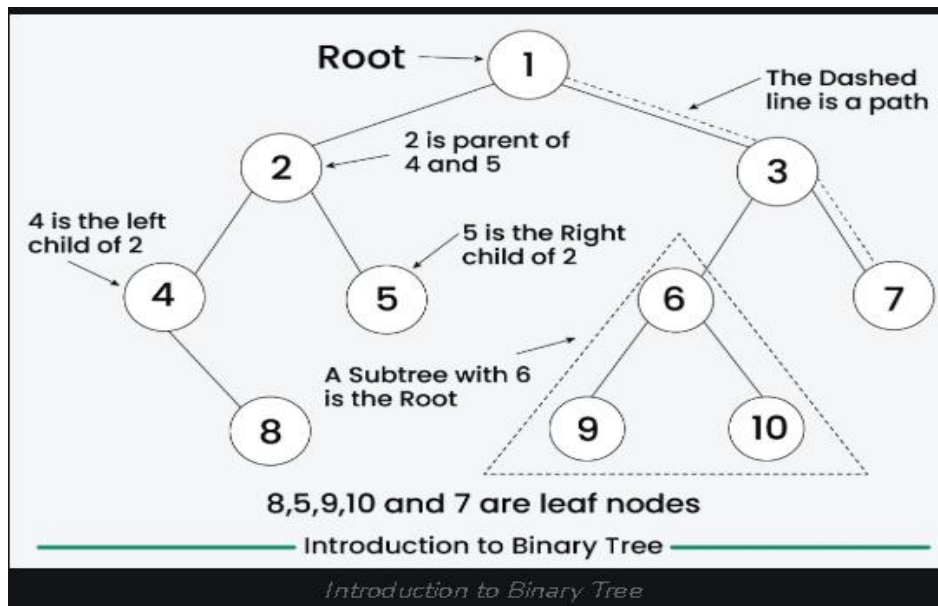
➤ Introduction to Tree Data Structure:

Tree data structure is a hierarchical structure that is used to represent and organize data in a way that is easy to navigate and search. It is a collection of nodes that are connected by edges and has a hierarchical relationship between the nodes.



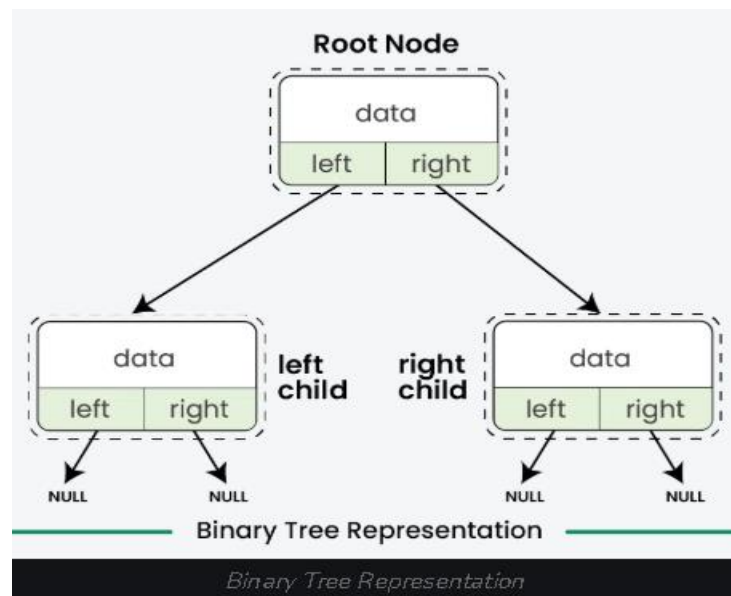
➤ Introduction to Binary Tree:

Binary Tree is a **non-linear** and **hierarchical** data structure where each node has at most two children referred to as the **left child** and the **right child**. The topmost node in a binary tree is called the **root**, and the bottom-most nodes are called **leaves**.



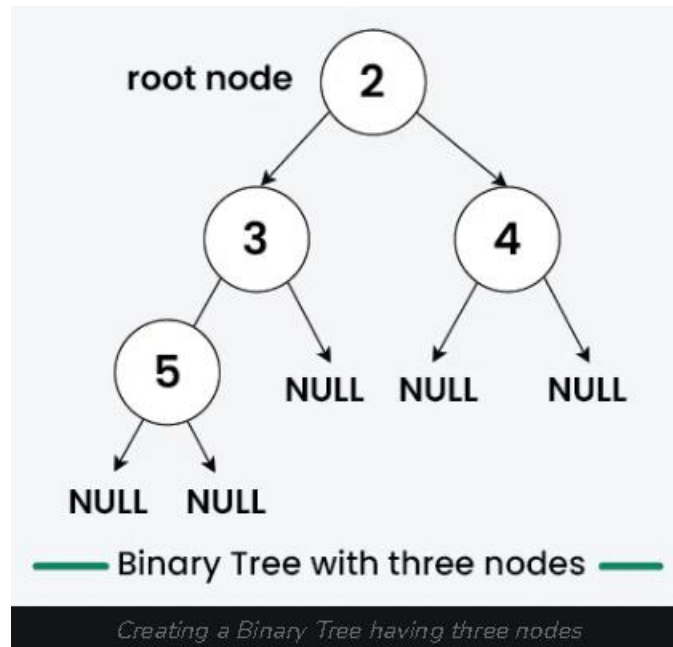
Representation of Binary Tree: Each node in a Binary Tree has three parts:

Data, Pointer to the left child, Pointer to the right child



Example for Creating a Binary Tree

Here's an example of creating a Binary Tree with four nodes (2, 3, 4, 5)



❖ Terminologies in Binary Tree

- I. **Nodes:** The fundamental part of a binary tree, where each node contains **data** and **link** to two child nodes.
- II. **Root:** The topmost node in a tree is known as the root node. It has no parent and serves as the starting point for all nodes in the tree.
- III. **Parent Node:** A node that has one or more child nodes. In a binary tree, each node can have at most two children.
- IV. **Child Node:** A node that is a descendant of another node (its parent).
- V. **Leaf Node:** A node that does not have any children or both children are null.
- VI. **Internal Node:** A node that has at least one child. This includes all nodes except the **root** and the **leaf** nodes.

- VII. **Depth of a Node:** The number of edges from a specific node to the root node. The depth of the **root** node is zero.
- VIII. **Height of a Binary Tree:** The number of nodes from the deepest leaf node to the root node.

➤ Binary Trees and Representation in Memory:

1. Dynamic (Linked) Representation

Binary trees are dynamically represented using pointers. Each node is allocated memory as required, and child nodes are linked via pointers.

Example: Building a Binary Tree

```
int main() {  
    // Creating nodes  
    Node* root = new Node(1); // Root node  
    root->left = new Node(2); // Left child of root  
    root->right = new Node(3); // Right child of root  
    root->left->left = new Node(4); // Left child of root->left  
    root->left->right = new Node(5); // Right child of root->left  
    // Output the tree structure  
    cout << "Root: " << root->data << endl;  
    cout << "Left child of Root: " << root->left->data << endl;  
    cout << "Right child of Root: " << root->right->data << endl;  
  
    return 0;  
}
```

Memory Layout:

- Each node has a **data field** and two **pointer fields** for its children.
- Memory is allocated dynamically during runtime using `new`.

2. Array Representation

In this method, a binary tree is represented as an array. The nodes are stored level-wise from top to bottom and left to right.

Formulae:

- For a node at index i :
 - **Left child:** $2 * i + 1$
 - **Right child:** $2 * i + 2$
 - **Parent:** $(i - 1) / 2$ (integer division)

Example: Array Representation

```
#include <iostream>
using namespace std;

int main() {
    // Binary tree represented as an array
    int tree[7] = {1, 2, 3, 4, 5, 6, 7};

    // Accessing elements
    cout << "Root: " << tree[0] << endl;
    cout << "Left child of Root: " << tree[1] << endl;
    cout << "Right child of Root: " << tree[2] << endl;

    return 0;
}
```

Memory Layout:

- The entire tree is stored in contiguous memory locations.
- Suitable for **complete binary trees**, but inefficient for sparse binary trees as unused elements waste memory.

Traversal in Binary Trees

Preorder Traversal (Root, Left, Right):

```
void preorderTraversal(Node* root) {
```

```

if (root == nullptr) return;
cout << root->data << " ";
preorderTraversal(root->left);
preorderTraversal(root->right);
}

```

Inorder Traversal (Left, Root, Right):

```

void inorderTraversal(Node* root) {
    if (root == nullptr) return;
    inorderTraversal(root->left);
    cout << root->data << " ";
    inorderTraversal(root->right);
}

```

Postorder Traversal (Left, Right, Root):

```

void postorderTraversal(Node* root) {
    if (root == nullptr) return;
    postorderTraversal(root->left);
    postorderTraversal(root->right);
    cout << root->data << " ";
}

```

Comparison: Linked vs. Array Representation

Aspect	Linked Representation	Array Representation
Flexibility	Dynamic, grows as needed	Static, fixed size
Memory Usage	Efficient for sparse trees	Inefficient for sparse trees
Ease of Access	Requires traversal via pointers	Direct access using indices
Implementation	Complex, uses dynamic memory	Simple, uses arrays

➤ Algorithm Design Techniques

1. Divide and Conquer

Binary trees inherently support recursive operations, making divide-and-conquer a natural technique. This involves solving the problem for the left and right subtrees and combining their results.

Example: Find the Height of a Binary Tree

```
#include <iostream>

using namespace std;

// Binary Tree Node Structure

struct Node {

    int data;

    Node* left;

    Node* right;

    Node(int value) {

        data = value;

        left = nullptr;

        right = nullptr;

    }

};

// Function to calculate the height of the binary tree

int height(Node* root) {

    if (root == nullptr) return 0; // Base case

    int leftHeight = height(root->left); // Recursive call for left subtree
```

```

    int rightHeight = height(root->right); // Recursive call for right subtree

    return max(leftHeight, rightHeight) + 1; // Combine results
}

int main() {

    // Creating a binary tree

    Node* root = new Node(1);

    root->left = new Node(2);

    root->right = new Node(3);

    root->left->left = new Node(4);

    root->left->right = new Node(5);

    // Calculate and display height

    cout << "Height of the tree: " << height(root) << endl;

    return 0;

}

```

2. Dynamic Programming on Trees

Dynamic programming (DP) is used to optimize problems with overlapping subproblems by storing intermediate results for each subtree.

Example: Find the Diameter of a Binary Tree

The diameter of a binary tree is the length of the longest path between any two nodes.

```

#include <iostream>

using namespace std;

struct Node {

    int data;

```

```

Node* left;

Node* right;

Node(int value) {
    data = value;
    left = nullptr;
    right = nullptr;
}

};

// Helper function to calculate the diameter
int diameter(Node* root, int& maxDiameter) {
    if (root == nullptr) return 0;

    int leftHeight = diameter(root->left, maxDiameter); // Left subtree height
    int rightHeight = diameter(root->right, maxDiameter); // Right subtree height
    maxDiameter = max(maxDiameter, leftHeight + rightHeight); // Update diameter
    return max(leftHeight, rightHeight) + 1; // Return height of current node
}

int findDiameter(Node* root) {
    int maxDiameter = 0;

    diameter(root, maxDiameter);

    return maxDiameter;
}

int main() {
    Node* root = new Node(1);

```

```

root->left = new Node(2);

root->right = new Node(3);

root->left->left = new Node(4);

root->left->right = new Node(5);

cout << "Diameter of the tree: " << findDiameter(root) << endl;

return 0;

}

```

3. Backtracking

Backtracking is useful for problems that explore all possible paths or configurations in a binary tree.

Example: Find All Root-to-Leaf Paths with a Given Sum

```

#include <iostream>

#include <vector>

using namespace std;

struct Node {

    int data;

    Node* left;

    Node* right;

    Node(int value) {

        data = value;

        left = nullptr;
    }
}

```

```

        right = nullptr;
    }
};

// Helper function to find paths
void findPaths(Node* root, int sum, vector<int>& path, vector<vector<int>>& result) {
    if (root == nullptr) return;
    path.push_back(root->data);
    sum -= root->data;
    // If a root-to-leaf path with the given sum is found
    if (root->left == nullptr && root->right == nullptr && sum == 0) {
        result.push_back(path);
    } else {
        findPaths(root->left, sum, path, result);
        findPaths(root->right, sum, path, result);
    }
    path.pop_back(); // Backtrack
}

int main() {
    Node* root = new Node(5);
    root->left = new Node(4);
    root->right = new Node(8);
    root->left->left = new Node(11);
    root->right->left = new Node(13);

```

```

root->right->right = new Node(4);

int targetSum = 22;

vector<int> path;

vector<vector<int>> result;

findPaths(root, targetSum, path, result);

cout << "Paths with sum " << targetSum << ":\n";

for (const auto& p : result) {

    for (int val : p) {

        cout << val << " ";

    }

    cout << endl;

}

return 0;

}

```

4. Greedy Approach

Greedy techniques are applied to problems where local optimization leads to a global solution.

Example: Check if a Binary Tree is a Binary Search Tree (BST)

```

bool isBST(Node* root, int minVal, int maxVal) {

    if (root == nullptr) return true; // Base case

    if (root->data <= minVal || root->data >= maxVal) return false;

    return isBST(root->left, minVal, root->data) && isBST(root->right, root->data, maxVal);

}

```

```
bool isBinarySearchTree(Node* root) {
    return isBST(root, INT_MIN, INT_MAX);
}
```

5. Breadth-First Search (BFS)

BFS is used for level-order traversal of the binary tree and is implemented using a queue.

Example: Level Order Traversal

```
#include <iostream>

#include <queue>

using namespace std;

void levelOrder(Node* root) {
    if (root == nullptr) return;
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        Node* current = q.front();
        q.pop();
        cout << current->data << " ";
        if (current->left != nullptr) q.push(current->left);
        if (current->right != nullptr) q.push(current->right);
    }
}
```

6. Depth-First Search (DFS)

DFS includes pre-order, in-order, and post-order traversals.

Example: Inorder Traversal

```
void inorder(Node* root) {  
    if (root == nullptr) return;  
    inorder(root->left);  
    cout << root->data << " ";  
    inorder(root->right);  
}
```

7. Memoization

Store results of subproblems to avoid redundant calculations.

Example: Count Unique Binary Search Trees

```
int numTrees(int n, vector<int>& dp) {  
    if (n <= 1) return 1;  
    if (dp[n] != -1) return dp[n];  
    int count = 0;  
    for (int i = 1; i <= n; i++) {  
        count += numTrees(i - 1, dp) * numTrees(n - i, dp);  
    }  
    dp[n] = count;  
    return count;  
}  
  
int countUniqueBSTs(int n) {  
    vector<int> dp(n + 1, -1);  
    return numTrees(n, dp);  
}
```


➤ Performance Analysis of Algorithms

Factors Influencing Performance

1. Tree Structure:

- a. **Balanced Tree:** The height of the tree is $O(\log n)$, where n is the number of nodes.
- b. **Unbalanced Tree:** The height can be as large as $O(n)$ in the worst case (e.g., skewed trees).

2. Traversal Type:

- a. Pre-order, in-order, post-order traversals involve visiting each node once.
- b. Level-order traversal requires additional space for a queue.

3. Node Count:

The time complexity often depends on the number of nodes (n).

➤ Types of Algorithm Analysis in Binary Trees Using C++:

When working with binary trees in C++, analyzing algorithms helps evaluate their performance in terms of **time complexity** and **space complexity**. Below are the common types of algorithm analysis applied in binary tree operations, including **Traversal**, **Insertion**, **Search**, and **Deletion**.

1. Traversal Analysis

a. Preorder Traversal

In preorder traversal, the nodes are visited in the order: **Root** → **Left** → **Right**.

Algorithm:

1. Visit the root node.
2. Recursively traverse the left subtree.
3. Recursively traverse the right subtree.

Time Complexity:

- **Best, Average, and Worst Case:** $O(n)$, where n is the number of nodes.
Each node is visited exactly once.

Space Complexity:

- **Recursive Implementation:** $O(h)$, where h is the height of the tree (stack space for recursive calls).
- **Iterative Implementation:** $O(h)$, using a stack to simulate recursion.

b. Inorder Traversal

In inorder traversal, the nodes are visited in the order: **Left $\rightarrow$ Root $\rightarrow$ Right**.

Algorithm:

1. Recursively traverse the left subtree.
2. Visit the root node.
3. Recursively traverse the right subtree.

Time Complexity:

- **Best, Average, and Worst Case:** $O(n)$, as each node is visited once.

Space Complexity:

- Recursive: $O(h)$.
- Iterative: $O(h)$.

c. Postorder Traversal

In postorder traversal, the nodes are visited in the order: **Left $\rightarrow$ Right $\rightarrow$ Root**.

Algorithm:

1. Recursively traverse the left subtree.
2. Recursively traverse the right subtree.
3. Visit the root node.

Time Complexity:

- $O(n)$: Each node is visited once.

Space Complexity:

- Recursive: $O(h)$.
- Iterative: $O(h)$.

d. Level Order Traversal

In level order traversal, nodes are visited level by level using a queue.

Algorithm:

1. Start with the root and enqueue it.
2. While the queue is not empty:
 - Dequeue a node, process it, and enqueue its children.

Time Complexity:

- $O(n)$: Every node is visited once.

Space Complexity:

- $O(w)$, where w is the maximum width of the tree (maximum nodes at any level).

2. Insertion Analysis

Insertion in a binary tree depends on the type of tree:

- **Binary Search Tree (BST)**: Insertion follows the property where the left child contains smaller values and the right child contains larger values.

Algorithm:

1. Start at the root.
2. Compare the value to be inserted with the current node.
3. Move left or right based on comparison until reaching a leaf.
4. Insert the new node at the correct position.

Time Complexity:

- **Best Case:** $O(1)$, when the tree is empty.
- **Average Case:** $O(\log n)$, for a balanced tree.
- **Worst Case:** $O(n)$, for a skewed tree.

Space Complexity:

- Recursive: $O(h)$.
- Iterative: $O(1)$.

3. Search Analysis

Searching involves traversing the tree to find a target value.

Algorithm:

1. Start at the root.
2. Compare the target with the current node value.
3. Move left or right based on comparison until the value is found or a leaf is reached.

Time Complexity:

- **Best Case:** $O(1)$, if the value is at the root.
- **Average Case:** $O(\log n)$, for a balanced tree.
- **Worst Case:** $O(n)$, for a skewed tree.

Space Complexity:

- Recursive: $O(h)$.
- Iterative: $O(1)$.

4. Deletion Analysis

Deletion in a binary tree involves three cases:

1. Node to delete is a leaf.
2. Node to delete has one child.
3. Node to delete has two children (replace with in-order predecessor or successor).

Algorithm:

1. Search for the node to delete.
2. Handle the three cases accordingly.

Time Complexity:

- **Best Case:** $O(\log n)$, for a balanced tree.
- **Worst Case:** $O(n)$, for a skewed tree.

Space Complexity:

- Recursive: $O(h)$.
- Iterative: $O(1)$.

Example Code Snippets**a. Preorder Traversal (Recursive)**

```
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* left;
    Node* right;
```

```

Node(int value) {
    data = value;
    left = right = nullptr;
}
};

void preorder(Node* root) {
    if (!root) return;
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
    preorder(root); // Output: 1 2 4 5 3
    return 0;
}

```

b. Insertion in a BST

```

Node* insert(Node* root, int key) {
    if (!root) return new Node(key);
    if (key < root->data)
        root->left = insert(root->left, key);
    else
        root->right = insert(root->right, key);
    return root;
}

```

➤ Order of Growth:

The **order of growth** in a binary tree refers to the time complexity and space complexity associated with common operations such as insertion, deletion, traversal, and searching. Here's a detailed breakdown:

1. Definitions

- **Binary Tree:** A tree data structure where each node has at most two children (referred to as the left child and the right child).
- **Height of Binary Tree (h):** The number of edges in the longest path from the root to a leaf.
- **Nodes (n):** The total number of nodes in the binary tree.

2. Types of Binary Trees

- **Full Binary Tree:** Every node has either 0 or 2 children.
- **Complete Binary Tree:** All levels are fully filled except possibly the last, which is filled from left to right.
- **Perfect Binary Tree:** All internal nodes have two children, and all leaves are at the same level.
- **Skewed Binary Tree:** Every parent node has only one child (either left or right).

3. Operations and Their Orders of Growth

A. Insertion

- **Average Case:** $O(\log n)$
 - If the tree is balanced (e.g., AVL or Red-Black Tree), the height is proportional to $\log n$.
- **Worst Case:** $O(n)$
 - For a skewed tree (all nodes are to one side), the insertion takes $O(n)$ because the height of the tree equals the number of nodes.

B. Deletion

- **Average Case:** $O(\log n)$

- Similar to insertion, balanced trees ensure $O(\log n)$ complexity.
- **Worst Case: $O(n)$**
 - In an unbalanced (skewed) tree, deletion may require traversing all nodes.

C. Searching

- **Average Case: $O(\log n)$**
 - In a balanced binary tree, searching involves following a path proportional to the height of the tree ($\log n$).
- **Worst Case: $O(n)$**
 - For a skewed tree, the search may traverse all nodes.

D. Traversal (Inorder, Preorder, Postorder)

- **Time Complexity: $O(n)$**
 - Every node is visited once.
- **Space Complexity: $O(h)$**
 - Due to the recursion stack (or explicitly maintained stack), the space complexity depends on the height of the tree.

E. Space Complexity

- **Tree Storage: $O(n)$**
 - Every node must be stored in memory.
- **Recursion Stack for Traversals: $O(h)$**
 - Depends on the height of the tree.

4. Relationship Between Height and Nodes

- For a balanced binary tree:

$$h = \log_2(n+1)$$

Time complexities remain logarithmic ($O(\log n)$).

- For a skewed binary tree:

$$h = n - 1$$

Time complexities degrade to linear ($O(n)$).

5. Special Cases

- **Balanced Binary Trees:** Ensure $O(\log n)$ time for most operations. Examples include AVL Trees and Red-Black Trees.
- **Degenerate (Skewed) Trees:** Behave like a linked list, with $O(n)$ time for most operations.

6. Summary Table

Operation	Balanced Tree	Skewed Tree
Insertion	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(n)$
Search	$O(\log n)$	$O(n)$
Traversal	$O(n)$	$O(n)$
Space Complexity	$O(n)$	$O(n)$

➤ Asymptotic Notations:

- I. Asymptotic Notations are mathematical tools used to analyze the performance of algorithms by understanding how their efficiency changes as the input size grows.
- II. These notations provide a concise way to express the behavior of an algorithm's time or space complexity as the input size approaches infinity.
- III. Rather than comparing algorithms directly, asymptotic analysis focuses on understanding the relative growth rates of algorithms' complexities.
- IV. It enables comparisons of algorithms' efficiency by abstracting away machine-specific constants and implementation details, focusing instead on fundamental trends.
- V. Asymptotic analysis allows for the comparison of algorithms' space and time complexities by examining their performance characteristics as the input size varies.
- VI. By using asymptotic notations, such as Big O, Big Omega, and Big Theta, we can categorize algorithms based on their worst-case, best-case, or average-case time or space complexities, providing valuable insights into their efficiency.

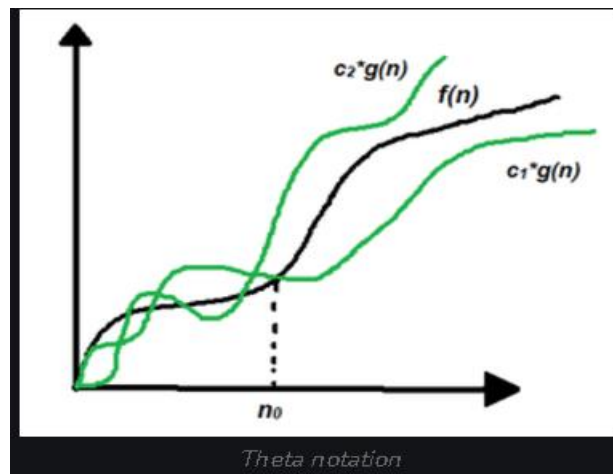
There are mainly three asymptotic notations:

1. Theta Notation (Θ -Notation):

Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm.

. Theta (Average Case) You add the running times for each possible input combination and take the average in the average case.

Let g and f be the function from the set of natural numbers to itself. The function f is said to be $\Theta(g)$, if there are constants $c_1, c_2 > 0$ and a natural number n_0 such that $c_1 * g(n) \leq f(n) \leq c_2 * g(n)$ for all $n \geq n_0$



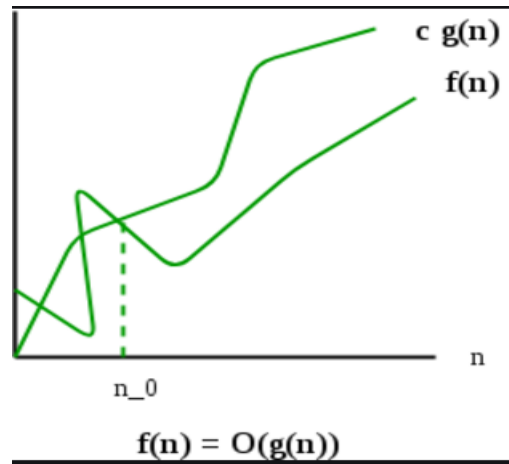
2. Big-O Notation (O-notation):

Big-O notation represents the upper bound of the running time of an algorithm. Therefore, it gives the worst-case complexity of an algorithm. It is the most widely used notation for Asymptotic analysis. It specifies the upper bound of a function. The maximum time required by an algorithm or the worst-case time complexity. It returns the highest possible output value(big-O) for a given input. Big-O (Worst Case) It is defined as the condition that allows an algorithm to complete statement execution in the longest amount of time possible.

If $f(n)$ describes the running time of an algorithm, $f(n)$ is $O(g(n))$ if there exist a positive constant C and n_0 such that, $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0$

It returns the highest possible output value (big-O) for a given input.

The execution time serves as an upper bound on the algorithm's time complexity.



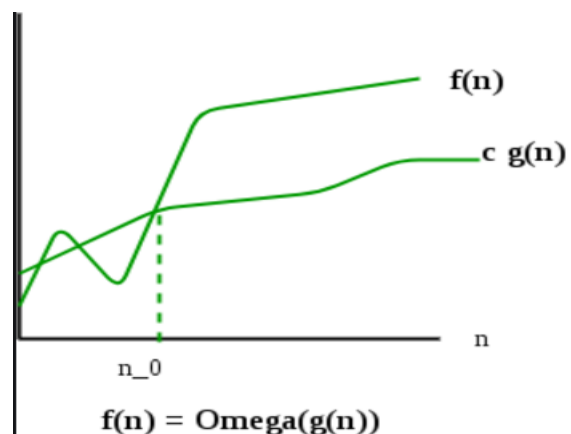
2. Omega Notation (Ω -Notation):

Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best-case complexity of an algorithm.

The execution time serves as a lower bound on the algorithm's time complexity.

It is defined as the condition that allows an algorithm to complete statement execution in the shortest amount of time.

Let g and f be the function from the set of natural numbers to itself. The function f is said to be $\Omega(g)$, if there is a constant $c > 0$ and a natural number n_0 such that $c \cdot g(n) \leq f(n)$ for all $n \geq n_0$.



Mathematical Representation of Asymptotic Notations

1. **Big-O Notation (O-notation):** Describes the **upper bound** of an algorithm's time or space complexity. It represents the **worst-case scenario**.

$$f(n)=O(g(n)) \Leftrightarrow \exists c>0, n_0>0 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \forall n \geq n_0$$

2. **Theta Notation (Θ -notation):** Represents the **exact bound** (tight bound) of an algorithm. The function grows at the same rate as $g(n)$.

$$f(n)=\Theta(g(n)) \Leftrightarrow \exists c_1, c_2>0, n_0>0 \text{ such that } c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \forall n \geq n_0$$

3. **Omega Notation (Ω -notation):** Describes the **lower bound** of an algorithm's time or space complexity. It represents the **best-case scenario**

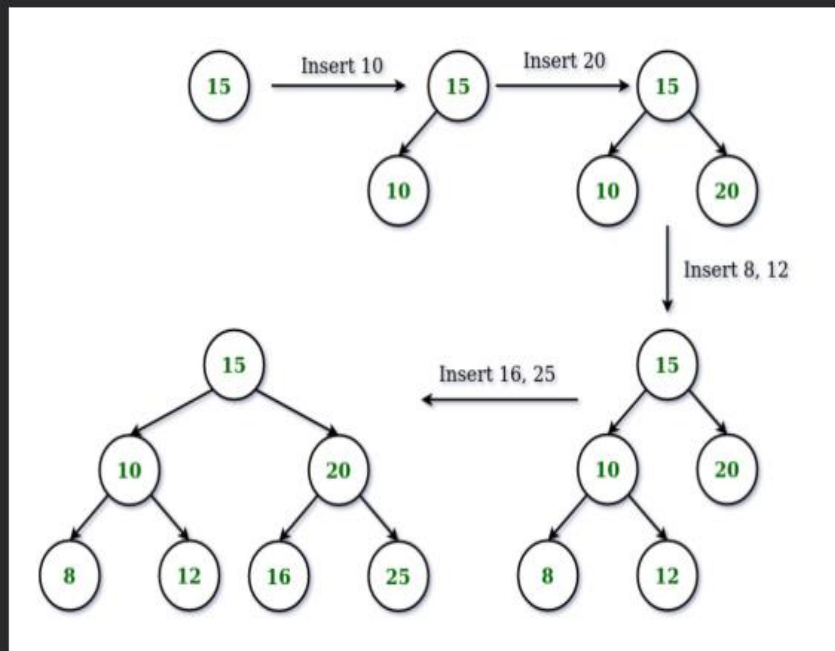
$$f(n)=\Omega(g(n)) \Leftrightarrow \exists c>0, n_0>0 \text{ such that } 0 \leq c \cdot g(n) \leq f(n) \forall n \geq n_0$$

❖ **Time complexity of a linear search in terms of O , Θ , and Ω .**

```
#include <iostream>
using namespace std;
bool linearSearch(int arr[], int n, int x) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == x)
            return true; // Element found
    }
    return false; // Element not found
}
int main() {
    int arr[] = {1, 3, 5, 7, 9};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 7;
    if (linearSearch(arr, n, x))
        cout << "Element found" << endl;
    else
        cout << "Element not found" << endl;
    return 0;
}
```

- ❖ Given an array of integers, the task is to construct a binary tree in level order fashion using **Recursion**.

Given an array `arr[] = {15, 10, 20, 8, 12, 16, 25}`



➤ Recurrence Relations in Binary Trees:

Recurrence relations are equations that describe the behavior of recursive algorithms or structures, such as binary trees. They express a problem in terms of smaller subproblems.

In the context of binary trees, recurrence relations are used to analyze:

1. **Number of nodes**
2. **Height of the tree**
3. **Traversals**
4. **Other recursive operations**

Key Recurrence Relations in Binary Trees

1. Number of Nodes in a Binary Tree

For a binary tree:

- **Relation:** A binary tree is composed of a root node and two subtrees (left and right).

If $T(n)$ is the total number of nodes in the tree:

$$T(n) = TL + TR + 1$$

Where:

TL : Number of nodes in the left subtree

TR : Number of nodes in the right subtree

2. Height of a Binary Tree

- If $H(n)$ is the height of a binary tree, the height depends on the maximum height of the left and right subtrees:

$$H(n) = \max(HL, HR) + 1$$

- Where:

- HL : Height of the left subtree

- HR : Height of the right subtree

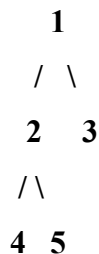
3. Traversal Time Complexity

- Traversal involves visiting each node, leading to:

$$T(n) = TL + TR + c$$

Here, c is the time to process the current node (usually $O(1)$).

❖ Example Walkthrough



Steps:

1. Call countNodes on root (1).

2. Recursively call countNodes on left subtree (root 2) and right subtree (root 3).
3. Compute for each subtree:
 - Subtree rooted at 2: 4,5,5 $\rightarrow T(2) = 3$
 - Subtree rooted at 3: No children $\rightarrow T(3) = 1$.
4. Combine: $T(1) = T(2) + T(3) + 1 = 3 + 1 + 1 = 5$.

➤ Substitution Method:

Explanation

1. TreeNode Structure:

- Each node has a value, a left pointer, and a right pointer.

2. Substitute Function:

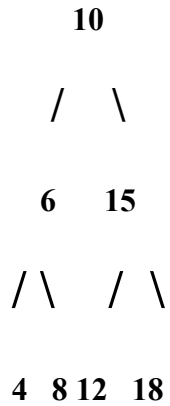
- If the node is null, return 0.
- If the node is a **leaf**, return its value.
- Otherwise, recursively compute the sum of the left and right subtrees.
- Replace the current node's value with this sum while preserving the original value for further calculations.

3. In-order Traversal:

- Used to display the binary tree values before and after substitution.

4. Main Function:

- Creates a binary tree.
- Prints its original in-order traversal.
- Substitutes node values using the recursive function.
- Prints the modified in-order traversal.



Original Tree (In-order): 4 6 8 10 12 15 18

Modified Tree (In-order): 0 12 0 62 0 30 0

```

#include <iostream>

using namespace std;

// Definition of a binary tree node

struct TreeNode {

    int value;

    TreeNode* left;

    TreeNode* right;

    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {}

};

// Function to substitute each node's value with the sum of its child nodes' values

int substituteValues(TreeNode* root) {

    // Base case: If the node is null, return 0

    if (!root) {

        return 0;

    }

```

```

// Base case: If the node is a leaf, return its value
if (!root->left && !root->right) {
    return root->value;
}

// Recursive calls to left and right subtrees
int leftSum = substituteValues(root->left);
int rightSum = substituteValues(root->right);

// Update the current node's value to the sum of left and right child values
int oldValue = root->value;
root->value = leftSum + rightSum;

// Return the updated sum including the current node's original value
return root->value + oldValue;
}

// In-order traversal to print the tree
void printInOrder(TreeNode* root) {
    if (!root) return;
    printInOrder(root->left);
    cout << root->value << " ";
    printInOrder(root->right);
}

int main() {
    // Creating a sample binary tree
    TreeNode* root = new TreeNode(10);

```



```

root->left = new TreeNode(6);
root->right = new TreeNode(15);
root->left->left = new TreeNode(4);
root->left->right = new TreeNode(8);
root->right->left = new TreeNode(12);
root->right->right = new TreeNode(18);
cout << "Original Tree (In-order): ";
printInOrder(root);
cout << endl;

// Perform substitution
substituteValues(root);
cout << "Modified Tree (In-order): ";
printInOrder(root);
cout << endl;

// Cleanup memory
delete root->left->left;
delete root->left->right;
delete root->right->left;
delete root->right->right;
delete root->left;
delete root->right;
delete root;

return 0;}

```

➤ Iterative Method:

1. Iterative Inorder Traversal

In this method, we use a stack to simulate the function call stack that is used in a recursive inorder traversal.

```
#include <iostream>
#include <stack>
using namespace std;
// Structure of a binary tree node
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(NULL), right(NULL) {}
};

// Iterative inorder traversal
void iterativeInorder(Node* root) {
    stack<Node*> s;
    Node* current = root;

    while (current != NULL || !s.empty()) {
        // Reach the leftmost node of the current node
        while (current != NULL) {
            s.push(current);
            current = current->left;
        }
        // Current must be NULL at this point
        current = s.top();
        s.pop();
        cout << current->data << " ";
        // Now, visit the right subtree
```

```

        current = current->right;
    }
}

int main() {
    // Example binary tree:
    //      1
    //     /\
    //    2 3
    //   /\
    //  4 5
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
    cout << "Inorder traversal: ";
    iterativeInorder(root);
    cout << endl;
    return 0;
}

```

2. Iterative Preorder Traversal

For preorder traversal, you can use a stack and first push the root node, then repeatedly pop the top of the stack, visit the node, and push its right and left children.

```

void iterativePreorder(Node* root) {
    if (root == NULL) return;
    stack<Node*> s;
    s.push(root);
    while (!s.empty()) {
        Node* current = s.top();
        s.pop();
        cout << current->data << " ";
    }
}

```

```

        // Push right child first so that left is processed first
        if (current->right) {
            s.push(current->right);
        }
        if (current->left) {
            s.push(current->left);
        }
    }
}

```

3. Iterative Postorder Traversal

Postorder traversal is a little trickier iteratively. One approach is to use two stacks. The first stack is used for processing the tree, and the second stack is used to reverse the order.

```

void iterativePostorder(Node* root) {
    if (root == NULL) return;
    stack<Node*> s1, s2;
    s1.push(root);
    while (!s1.empty()) {
        Node* current = s1.top();
        s1.pop();
        s2.push(current);
        // Push left and right children to s1
        if (current->left) {
            s1.push(current->left);
        }
        if (current->right) {
            s1.push(current->right);
        }
    }

    // Print the nodes in postorder
}

```

```

while (!s2.empty()) {
    cout << s2.top()->data << " ";
    s2.pop();
}
}

```

➤ Recursion Tree:

The Recursion Tree Method is a way of solving recurrence relations. In this method, a recurrence relation is converted into recursive trees. Each node represents the cost incurred at various levels of recursion. To find the total cost, costs of all levels are summed up.

Steps to solve recurrence relation using recursion tree method:

1. Draw a recursive tree for given recurrence relation
2. Calculate the cost at each level and count the total no of levels in the recursion tree.
3. Count the total number of nodes in the last level and calculate the cost of the last level
4. Sum up the cost of all the levels in the recursive tree

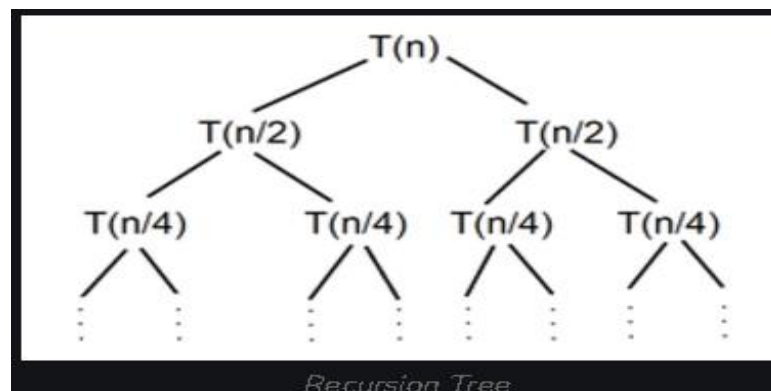
Note: If summing up all the levels becomes complex, we can find an upper bound by considering a perfectly full tree and / or an infinite geometrical series (the ratio is typically less than 1).

Question 1:

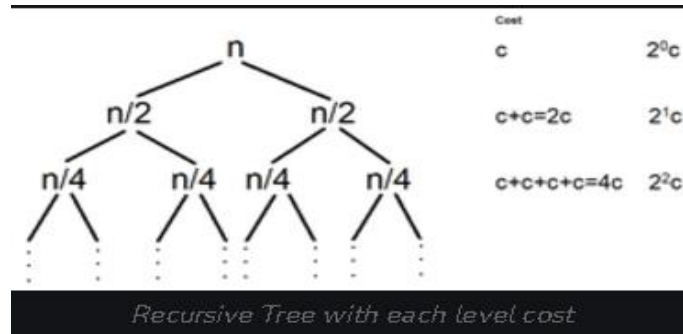
$$T(n) = 2T(n/2) + c$$

Solution:

- **Step 1:** Draw a recursive tree



- **Step 2:** Calculate the work done or cost at each level and count total no of levels in recursion tree



Count the total number of levels –

Choose the longest path from root node to leaf node

$$n/2^0 \rightarrow n/2^1 \rightarrow n/2^2 \rightarrow \dots \rightarrow n/2^k$$

Size of problem at last level = $n/2^k$

At last level size of problem becomes 1

$$n/2^k = 1$$

$$2^k = n$$

$$k = \log_2(n)$$

Total no of levels in recursive tree = $k + 1 = \log_2(n) + 1$

- **Step 3:** Count total number of nodes in the last level and calculate cost of last level

No. of nodes at level 0 = 1

No. of nodes at level 1 = 2

.....

No. of nodes at the $(k+1)$ th level (or last level) = $2^k = 2^{\log_2(n)} = n$

Cost of sub problems at level last level = $n \times \Theta(1) = \Theta(n)$

- **Step 4:** Sum up the cost all the levels in recursive tree

$T(n) = c + 2c + 4c + \dots ((\text{no. of levels}-1) \text{ times}) + \text{last level cost}$

$$= c + 2c + 4c + \dots (\log_2(n) \text{ times}) + \Theta(n)$$

$$= c (1 + 2 + 4 + \dots (\log_2(n) \text{ times})) + \Theta(n)$$

$1 + 2 + 4 + \dots (\log_2(n) \text{ times}) \rightarrow 2^0 + 2^1 + 2^2 + \dots (\log_2(n) \text{ times}) \rightarrow \text{Geometric Progression (G.P.)}$

Using GP Sum Formula, we get the sum of the series as n

$$= c(n) + \Theta(n)$$

Thus, **$T(n) = \Theta(n)$**

➤ **Master Theorem:**

In a binary search tree, each node has at most two children: left and right. The left child contains a value smaller than the parent node, while the right child contains a value greater than the parent node. Operations on a binary search tree generally have recursive characteristics, which makes them well-suited for the Master Theorem.

Operations on BST:

The most common operations on a BST include:

1. **Search**
2. **Insertion**
3. **Deletion**

All of these operations rely on a recursive structure, where each recursive call processes a subtree and either moves to the left or right child based on the value being searched for, inserted, or deleted.

Recurrence Relation for BST Operations:

The time complexity of operations like search, insertion, and deletion can be represented by the following recurrence relation:

$$T(n)=T(n/2)+O(1)$$

Where:

- $T(n)$ is the time complexity to process nn nodes in the tree.
- $T(n/2)$ represents the time complexity of processing the left or right subtree (assuming the tree is balanced).
- $O(1)$ represents the time spent on the current node (such as comparison or pointer assignment).

This recurrence arises because for each recursive step, you split the problem in half (like traversing down the left or right child of a node) and do constant work.

Applying the Master Theorem:

The recurrence relation for a typical BST operation can be written in the form:

$$T(n)=aT(n/b)+O(nd)$$

Where:

- aa is the number of subproblems.
- bb is the factor by which the problem size is reduced in each subproblem.
- dd is the exponent of the work done at each level of recursion.

For **binary search tree operations**, the recurrence is typically:

$$T(n)=T(n/2)+O(1)$$

So, here:

- $a=1$ (only one recursive call is made)
- $b=2$ (problem size is reduced by half)
- $d=0$ (constant work at each level)

Using the Master Theorem, we check the three cases:

1. If $a < b^d$, the complexity is $O(n^d)$.
2. If $a = b^d$, the complexity is $O(n^d \log n)$.
3. If $a > b^d$, the complexity is $O(n \log b^a)$.

For our case:

- $a=1$
- $b=2$
- $d=0$

We find that $a = b^d$, so we are in **Case 2** of the Master Theorem, and the complexity is:

$$T(n)=O(\log n)$$